Document number: P0480R0

Ville Voutilainen
2016-10-15

Explicit type checking with structured bindings

Abstract

Structured bindings in C++17 deduce the type of the incoming entity, and bind references to its elements. The types of the element bindings cannot be specified; this is a problem for large-scale programs where it would be desirable to be able to write code where it's possible to express the intent to use a binding of a specific type. There are library solutions to the problem, but they are incomplete and cumbersome.

Contents

Inconsistency

With C++ amended with concepts, it's possible to declare variables that are

1. of specific type
2. of a type deduced from an initializer
3. of a constrained type deduced from an initializer.

Similarly, it's possible to declare function parameters that are

1. of specific type
2. of a type deduced from a call expression's arguments
3. of a constrained type deduced from a call expression's arguments.

Similarly, it's possible to declare function return values that are

1. of specific type
2. of a type deduced from a return statement
3. of a constrained type deduced from a return statement.

Finally, it's possible to declare lambda captures that are

1. of specific type (by using an explicit conversion in an init-capture)
2. of a constrained type deduced from an init-capture's initializing expression
3. of a type deduced from the type of the captured entity
4. of a constrained type deduced from the type of the captured entity.

However, for a structured binding, the type of a binding can be

1. the type deduced from the element type of the entity for which the bindings are declared.

There is no simple way to specify the expected type of a binding established by structured bindings directly when declaring the binding.

What is the problem?

Well, despite what the Almost Always Auto proponents extol the benefits of using auto everywhere to be, there are very good reasons to write code that does something like

```
SpecificType var = func();
... /* some code in between */
process(var);
```

Reasoning about that code is fairly straightforward. We know that the type of var must be SpecificType, and we know that what func() returns must be convertible to SpecificType. If the API of func() changes in an incompatible way, we know about that at the point of declaration of var, and we don't even need to see whether process(var) became ill-formed, and we certainly don't need to worry about whether that call changed meaning. We have established a strongly-typed contract between func and the calling code, so that there's no duck typing involved, and we can rely on incompatible changes to the API of func() being loud and to notice them early.

In contrast, if we have something like

```
auto var = func();
... /* some code in between */
process(var);
```

we will not see any incompatible changes to the API of func() until we use the result, in process(var). We have made a trade-off from explicit precise typing to duck typing. That trade-off may well be a good idea, but there are a lot of cases where such a trade-off isn't the right thing to do.

With structured bindings, what we have is

```
auto [var, var2] = func();
... /* some code in between */
process(var, var2);
```

The only option we have is duck typing. No concrete types for the bindings, and not even constrained types to restrict the kind of types we expect. If the element types of the entity returned by func() change, we will not notice that before process(), and we might notice at all before we debug the program.

Fine, just static_assert the types you expect

One way to avoid the problem is to add a check for the types after the binding declaration:

```
auto [var, var2] = func();
static_assert(is_same_v<decltype(var), SpecificType>);
... /* some code in between */
process(var, var2);
```

This is not so simple in a loop:

```
for (auto [var, var2] : func()) {
  static_assert(is_same_v<decltype(var), SpecificType>);
  ... /* some code in between */
  process(var, var2);
}
```

Why is my start_assert inside the loop? It's not dependent on the values of the bound variables. I don't think I can write it in the loop header, because there's no place where the bindings would be in scope where I could put the static_assert.

Enter ensure

Peter Dimov showed an example of a library function that can check the elements of the target of the structured bindings. Its use looks roughly like this:

```
auto [var, var2] = ensure<SpecificType, SpecificType2>(func());
... /* some code in between */
process(var, var2);
```

It also works just fine in a loop:

```
for (auto [var, var2] : ensure<SpecificType, SpecificType2>(func())) {
  ... /* some code in between */
  process(var, var2);
}
```

Great, we have solved the problem, right?

Two questions come to mind when seeing such an ensure():

1. How does it cope with aggregate structs that don't have tuple_size and get<> customizations points?
2. How does it cope with xvalues and prvalues when the function called returns a reference?

There's a separate proposal that proposes having tuple_size and get<> just work for such aggregate structs, so we are not going to spend more time on that part here. However, the value category is interesting, thrilling and blood-chilling.

Code like this is always fine:

```
auto [var, var2] = ensure<SpecificType, SpecificType2>(func());
... /* some code in between */
process(var, var2);
```

In that code, func() may have return a reference or a temporary, either way it will work even if passed through ensure(), which takes a universal reference and returns it. The binding accepts its source object by value, so no temporary was destroyed before the binding happened and everything is fine.

Let's consider something different:

```
auto&& [var, var2] = ensure<SpecificType, SpecificType2>(func());
... /* some code in between */
process(var, var2);
```

Now, if func() returns by value, ensure manages to turn that prvalue into an xvalue, and the temporary returned by func() drops dead after the bindings are made (presumably not before), since its lifetime wasn't extended, and our bindings end up being invalid.

It seems tricky to solve this problem. Whenever ensure takes a reference and returns a reference, it manages to turn a prvalue into an xvalue, breaking lifetime extension.

Well, can't we just have ensure return by value, and rely on mandatory elision to do away with the extra move? No. That doesn't work if the function we call returns a reference, because that breaks identity.

Ensure on steroids

Barry Revzin suggested a different approach; have ensure call a function object. That way ensure can return what the function object returns, and there's no prvalue-to-xvalue conversion and no breaking identity. That looks roughly like this:

```
auto&& [var, var2] = ensure_func<SpecificType, SpecificType2>([]() -> decltype(auto) {return func();});
... /* some code in between */
process(var, var2);
```

The caller must naturally understand to use a lambda with a decltype(auto) return type, which is also the new return type of ensure_func.

We can make it "prettier", by desperately using a macro:

```
#define EVIL(X) []() -> decltype(auto) {return X;}
auto&& [var, var2] = ensure_func<SpecificType, SpecificType2>(EVIL(func());
... /* some code in between */
process(var, var2);
```

Should we have a language extension?

So, with the ingredients of a library helper backed up with a language extension that allows tuple_size and get<> to just work on aggregate structs, mixed with a lambda and possibly a macro, I can get what I want. It's not pretty, but I can make it work.

- Do we want users to need to rely on such techniques if they want explicit typing of their bindings?
- Do we consider such wantings a problem worth solving?
- How do we think such problems should be solved?